

AvanCha Kiran Kumar

20233093

CSE-A2

Q1)

man → to see manual

\$ man ls

\$ man cd

ls → list file and directory

\$ ls

\$ ls |

cd → change working directory

\$ cd ..

\$ cd ./DirectoryName

pwd → print working Directory

\$ pwd

mkdir → make directory

\$ mkdir DirectoryName

rmdir → remove directory

\$ rmdir ./DirectoryName

cat → display file content

\$ cat file.txt

less → pages program used to view the content

\$ less

head → display ^{few} lines of a file

\$ head -v

\$ head -n 10

tail → display few lines from last of a file

\$ tail -n 2

Avancho Kiran Kumar
20233093
CSE-A2

cp → copy the file src → dist

\$ man cp

\$ cp

\$ cp -L

mv → move file

\$ mv t.txt DirectoryName

rm → remove file or directory

\$ rm t.txt

touch → create an empty file

\$ touch text.txt

echo → out the string that is parsed

\$ echo "Helloworld"

\$ echo text.txt

ping →

\$ ping google.com

chmod → changes access mode of a file or directory

\$ man chmod

\$ chmod -w hello.txt

chown → change ownership

\$ chown root hello.txt

exit → terminates the current program

\$ exit

AvanCha Kiran Kumar

20233093

CSE-A2

Grep → Search pattern in a file

```
$ grep "hello"
```

```
$ grep -i "hello"
```

```
$ grep -e [xd]"00"
```

diff → show differences between two files

```
$ diff hello1.txt hello2.txt
```

ps → Show working process

```
$ ps
```

```
$ ps -f
```

```
$ ps -p 4000
```

top → table of process

Kill → Kill process

Sudo → run as admin

Shutdown → shutdowns the system

wc → word count

```
$ wc hello.txt
```

Avanah Kiran Kumar

20233093

A2

Q2) ① cpu info

```
Cat /proc/cpuinfo
```

```
Cat /proc/cpuinfo | grep "model name"
```

```
Cat /proc/cpuinfo | grep "cpu core"
```

```
Cat /proc/cpuinfo | grep "cpu cores"
```

② memory info

```
Vm stat
```

```
free -m
```

```
Cat /proc/meminfo
```

Q5)

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <time.h>
```

```
#include <limits.h>
```

```
void swap (int *a, int *b) {
```

```
    int temp = *a;
```

```
    *a = *b;
```

```
    *b = temp;
```

```
}
```

```
void selectionSort (int arr[], int n) {
```

```
    static clock_t t; for (int i=0; i<n-1; i++) {
```

```
        int mini = i;
```

```
        for (int j=i+1; j<n; j++) {
```

```
            if (arr[j] < arr[mini]) mini = j;
```

```
        }
```

```
        if (mini != i) swap(arr[mini], arr[i]);
```

```
    }
```


AvanCha Kishan Kumar

20233093

CSE-A2

```
end = clock();
```

```
int time = clock;
```

```
double time = end - start;
```

```
printf("Time of execution of selection sort: %f",  
time);
```

```
}
```

```
void BubbleSort(int arr[], int n) {
```

```
start = clock();
```

```
for (int i = 0; i < n - 1; i++) {
```

```
for (int j = 0; j < n - i - 1; j++) {
```

```
if (arr[j] > arr[j + 1]) {
```

```
swap(&arr[j], &arr[j + 1]);
```

```
}
```

```
}
```

```
}
```

```
end = clock();
```

```
double time = end - time;
```

```
printf("Execution time for Insertion Sort: %f", time);
```

```
}
```

```
void InsertionSort(int arr[], int n) {
```

```
start = clock();
```

```
for (int i = 1; i < n; i++) {
```

```
int j = i - 1;
```

```
for (j = i - 1; j >= 0; j--) {
```

```
while (j >= 0 && arr[j] > arr[j + 1]) {
```

```
arr[j + 1] = arr[j];
```

```
}
```

```
j--;
```

Avanchar Kisan Kumar

20233093

CSE-A2

```
arr[i+1] = arr[i];  
}  
end = clock();  
double time = end - start;  
printf("Execution Time for Insertionsort: %lf", time);  
}
```

Q3) (i) cd

```
#include <stdio.h>  
#include <sys/stat.h>  
#include <sys/types.h>  
#include <syslib.h>
```

```
int main (int args, char **args) {  
    if (args != 2) {  
        printf("Usage: %s <dir.name>Xn",  
               args[0]);  
        return 1;  
    }  
    int status = dir(args[1]);  
    if (status != -1) printf("%s directory changed",  
                             args[1]);  
    else printf("%s directory not changed",  
               char *cwd = getcmd(NULL, 0);
```


Avanah Kiran Kumar

20233093

CSE-A2

```
if (cwd = NULL) {  
    printf("get cwd failed");  
    return 1;  
}  
else {  
    printf("%s directory not created", args[1]);  
}  
return 0;  
}
```

(ii) IS

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#include <sys/types.h>
```

```
#include <dirent.h>
```

```
int main() { int argc, char ** args; }
```

```
DIR * dir;
```

```
struct dirent * entry;
```

```
bool list = false;
```

```
char * fmt = // usage [dir.pathname]
```

```
if (argc ≥ 2 || args[1][0] == '-') {
```

```
    int len = strlen(args[1]);
```

```
    if (len ≥ 2) {
```

```
        if (args[1][1] == 'G') show-all=true;
```

```
        else if (args[1][1] == 'l') list=true;
```

```
    } else {
```

```
        printf(fmt, args[0]);
```

```
        return 2;
```

```
    }
```

```
    if (len ≥ 3) {
```

```
        if (args[1][2] == 'a') show-all=true;
```

```
        else if (args[1][2] == 'l') list=true;
```

AvanCha Kiran Kumar

20233093

CSE-A2

```
else {
```

```
    printf(fmt, args[0]);
```

```
    return 2;
```

```
}
```

```
}
```

```
if (argc <= 1 || args[args[argc]-1][0] == '-') {
```

```
    dir = opendir(".");
```

```
    dir = opendir(args[args-1]);
```

```
    return 3;
```

```
}
```

```
while (entry = readdir(dir)) != NULL {
```

```
    if (!showall && entry->dir_name[0] == '.') {
```

```
        continue;
```

```
        printf(" %s", entry->d.name);
```

```
        if (list) printf(" ");
```

```
}
```

```
printf("\n");
```

```
closedir(dir);
```

```
return 0;
```

```
}
```

(iii) MKDIR

```
#include <stdio.h>
```

```
#include <sys/stat.h>
```

```
#include <sys/types.h>
```

```
int main (int argc, char **args) {
```

```
    if (argc < 2) {
```

```
        printf(" Usage: mkdir <dir.name>");
```

```
        return 1;
```

```
}
```


AvanCha Kiran Kumar

20233093

CSE-A2

```
int states = mkdir (args [i]).S_IRWX-U);  
if (states != -1) {  
    printf (" %s directory created \n", args [i]);  
}
```

(iv) Grep

```
#include <stdio.h>  
#include <stdlib.h>  
#include <string.h>  
  
void Grep (const char * filename, const char * keyword) {  
    FILE * file = fopen (filename, "r");  
    if (file == NULL) {  
        printf ("Error opening file:");  
        exit (EXIT_FAILURE);  
    }  
    char line [MAX-GIVE];  
    int line number = 0;  
    while (fgets (line, MAX-GIVE, file)) {  
        line number++;  
        if (strstr (line, keyword))  
            printf ("line %d %s \n", line number, line);  
    }  
    fclose (file);  
}  
  
int main (int args, char ** args) {  
    if (arg != -1) {  
        printf ("Usage: %s  
        <filename> <keyword>")  
        Grep (argv [1], argv [2]);  
    }  
    return 0;
```


Avanchar Kiran Kumar

20233093

CSE-A2

Q4) The above C program demonstrates the use of Command-line Arguments. This program calculates the sum of integers passed as command-line arguments.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main (int argc, char*argv[]) {
```

```
    if (argc < 2) {
```

```
        printf("Usage: %s <number1> <number2> ...",
```

```
              argv[0]);
```

```
    }
```

```
    int sum = 0;
```

```
    for (int i = 1; i < argc; i++) {
```

```
        sum += atoi(argv[i]);
```

```
    }
```

```
    printf("The sum of the numbers is: %d\n", sum);
```

```
    return 0;
```